Angelica Dias Folgado

RU: 4937552

RAÍZES DO NORDESTE

Projeto de API REST - Back-end

Implementação Java 21 / Spring Boot

Relatório acadêmico e evidências técnicas

2026

Apresentação

Angelica Dias Folgado - RU 4937552

Projeto acadêmico: Raízes do Nordeste

API REST para pedidos multicanal, estoque por unidade e pagamento simulado

Relatório técnico da implementação Java 21 e Spring Boot 3.5.

Disciplina: Projeto Multidisciplinar - Trilha Back-end. Curso, polo, semestre e docentes não informados não foram presumidos.

A discente designou os documentos locais da pasta uninter como referência oficial para esta entrega. Os arquivos contêm aviso de edição para estudo, conteúdo omitido e prevalência do original; esta adoção registra a orientação da discente, sem autenticação independente no AVA.

O sistema é uma demonstração acadêmica local. Pagamentos são simulados e os resultados de desempenho não constituem garantia para produção.

Sumário

1. Introdução

2. Estudo de caso e escopo

3. Análise e requisitos

4. Modelagem e arquitetura

5. API e endpoints

6. Segurança, privacidade e limites

7. Entrega técnica e aderência à trilha

8. Plano de testes e evidências

9. Ensaio de desempenho

10. Conclusão e referências

Anexos: diagramas do banco, casos de uso, arquitetura Java e sequência.

1. Introdução

Uma rede de alimentação precisa receber pedidos por diferentes canais, consultar a disponibilidade de cada unidade e acompanhar o preparo. O projeto implementa uma API para demonstrar esse fluxo com persistência e controle de acesso.

O objetivo é demonstrar o fluxo Pedido -> Pagamento simulado -> Status com banco real, autenticação e regras de autorização. Os canais APP, WEB, TOTEM, BALCAO e PICKUP são atributos do pedido.

2. Estudo de caso e escopo

A matriz acompanha a rede; cada unidade mantém saldo próprio por produto. ADMIN representa a administração da rede. O sistema não implementa interface gráfica nem integração financeira externa.

Foi escolhido o Fluxo A do material de referência: criar pedido, validar itens e saldo, registrar resultado de pagamento mock e atualizar status. O preço vem do banco, itens repetidos são agregados e o titular do pedido vem do JWT.

Cadastro público, administração completa de produtos/unidades, entradas administrativas de estoque, pontos e campanhas são propostas fora do MVP.

3. Análise e requisitos

A API autentica usuários por e-mail e senha e emite JWT. Senhas são armazenadas com BCrypt.

As permissões distinguem CLIENTE, ATENDENTE, COZINHA e ADMIN. Clientes consultam apenas seus pedidos; operações de cozinha e administração seguem regras de perfil.

O catálogo informa produtos e estoque da unidade selecionada. Listagens são paginadas, com limite máximo de 100 registros por página.

Criar pedido, reservar estoque e registrar auditoria ocorre em transação. Itens repetidos são agrupados; falta de saldo ou falha reverte a operação.

O pagamento mock aceita aprovação, recusa e falhas técnicas simuladas. Aprovação move o pedido para EM_PREPARO; a cozinha pode avançar para PRONTO e ENTREGUE.

Pedidos não pagos podem ser cancelados por perfis autorizados, com devolução transacional do estoque. Consentimento de fidelidade é demonstrativo, opcional e revogável.

Requisitos não funcionais atendidos incluem separação por responsabilidades, persistência SQLite, JWT, hashing de senha, padrão JSON de erro, limite de corpo HTTP de 100 KB, paginação e proteção de reserva sob concorrência.

Requisitos propostos: cadastro, CRUD administrativo de catálogo, movimentação de estoque, fidelidade por pontos, promoções e persistência distribuída multiwriter.

4. Modelagem e arquitetura

A aplicação foi implementada em Java 21 com Spring Boot 3.5, API REST, JDBC e SQLite. A documentação OpenAPI está disponível em /api-docs.

ApiController atende HTTP; AuthenticationFilter e JwtService validam identidade; RaizesStore centraliza JDBC e transações; PaymentGateway é a porta de pagamento e MockPaymentGateway seu adaptador local; ApiExceptionHandler projeta erros JSON.

O banco tem nove tabelas: usuarios, unidades, produtos, estoques_unidades, pedidos, itens_pedido, pagamentos, audit_logs e consentimentos. O par unidade/produto do estoque é único; pedidos têm usuário e unidade; itens referenciam pedido e produto; há no máximo um pagamento por pedido.

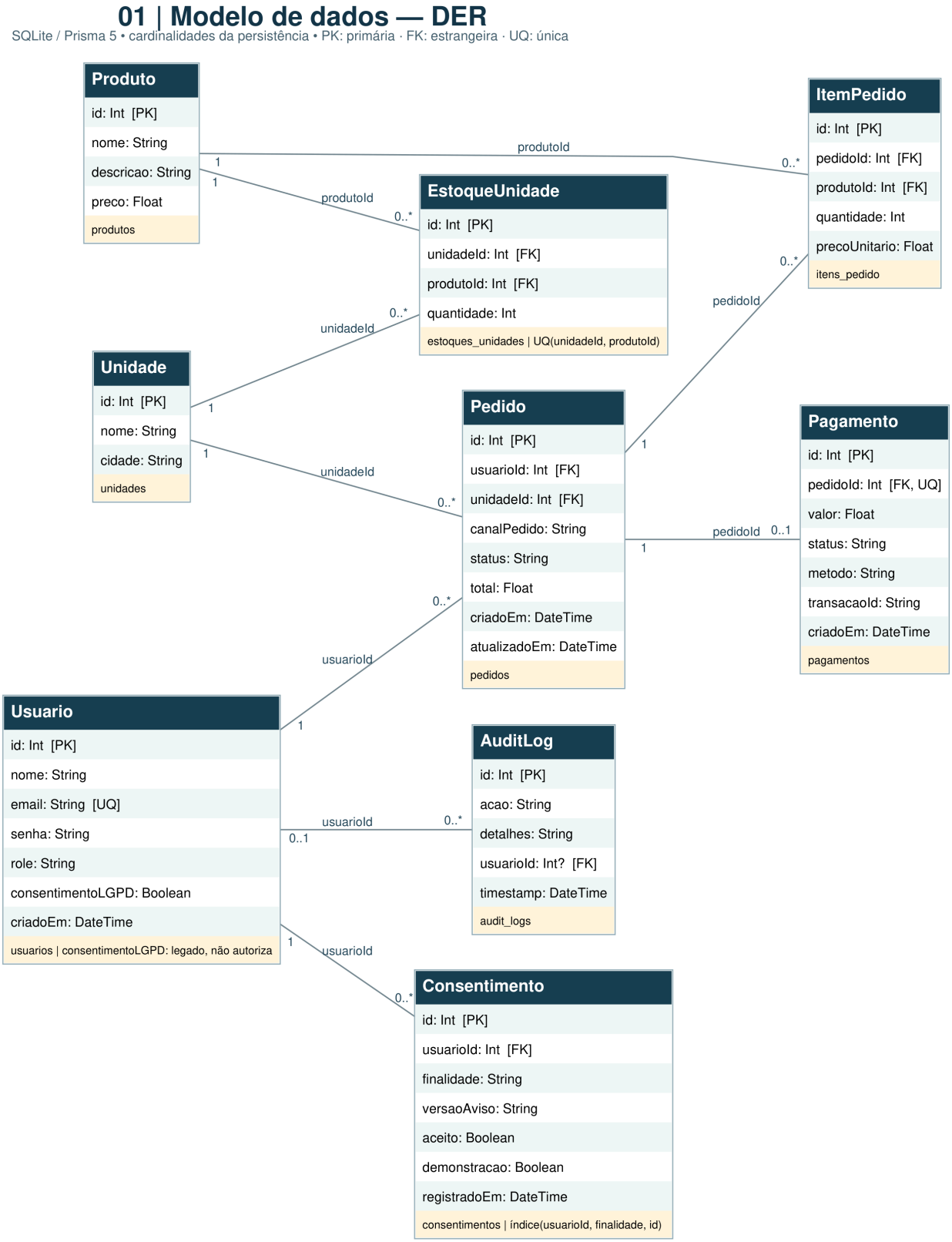
O código Java usa mapas JDBC e records pontuais; não possui classes Java de entidade para todo o domínio. O diagrama conceitual de classes preservado é histórico; o anexo de arquitetura Java registra as classes técnicas da implementação atual.

SQLite permite um escritor por vez. Uma trava local serializa transações de escrita dentro do processo; isso não coordena múltiplas instâncias distribuídas.

A implementação da API, dos testes automatizados e das ferramentas mantidas no projeto é Java 21. A afirmação 100% Java refere-se a esse código-fonte executável; documentação, configuração, SQL e artefatos de integração permanecem em seus formatos próprios.

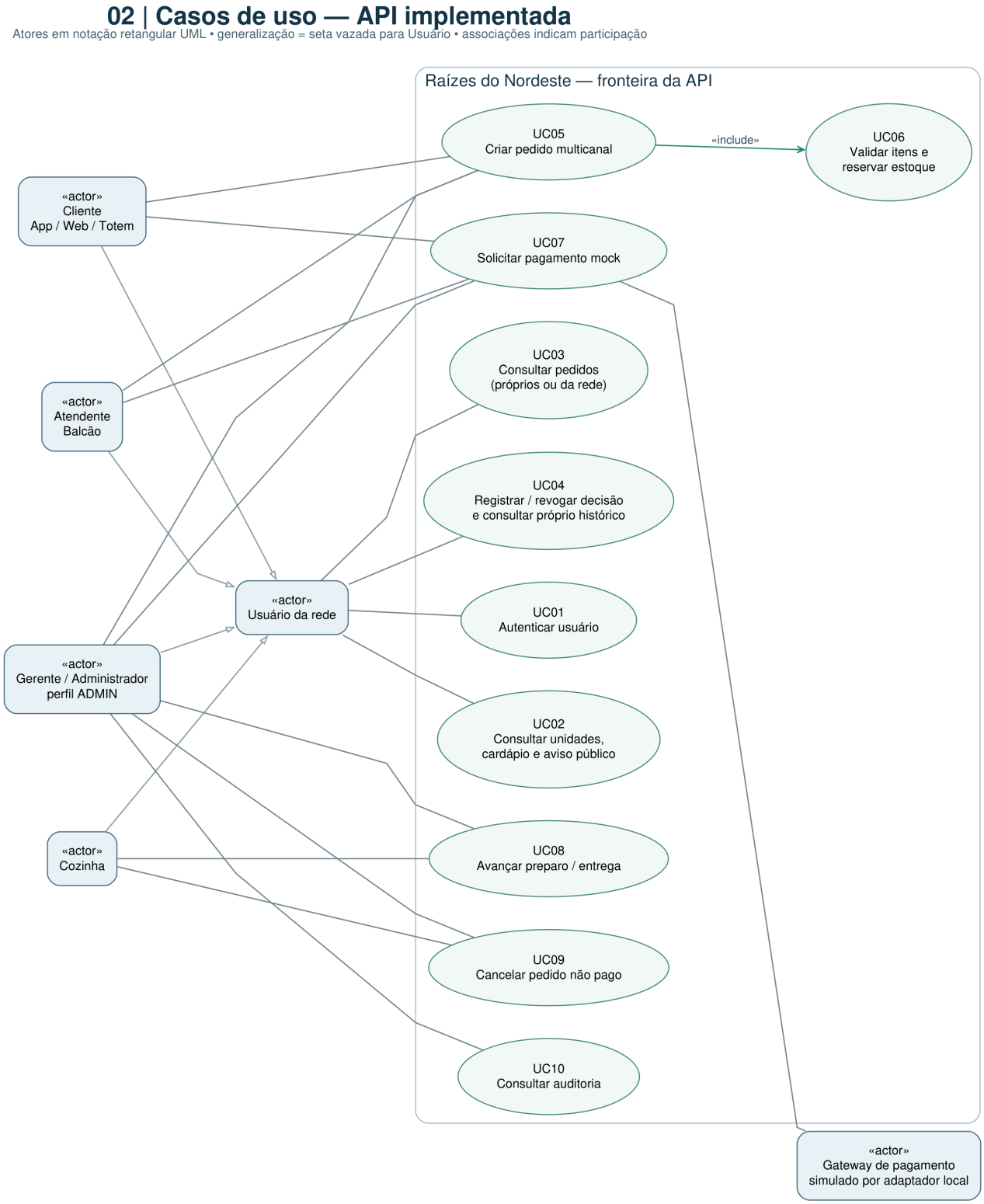
Anexo A - Diagrama entidade-relacionamento

Nove tabelas e relacionamentos. schema.sql é a fonte do modelo persistido pela API Java.

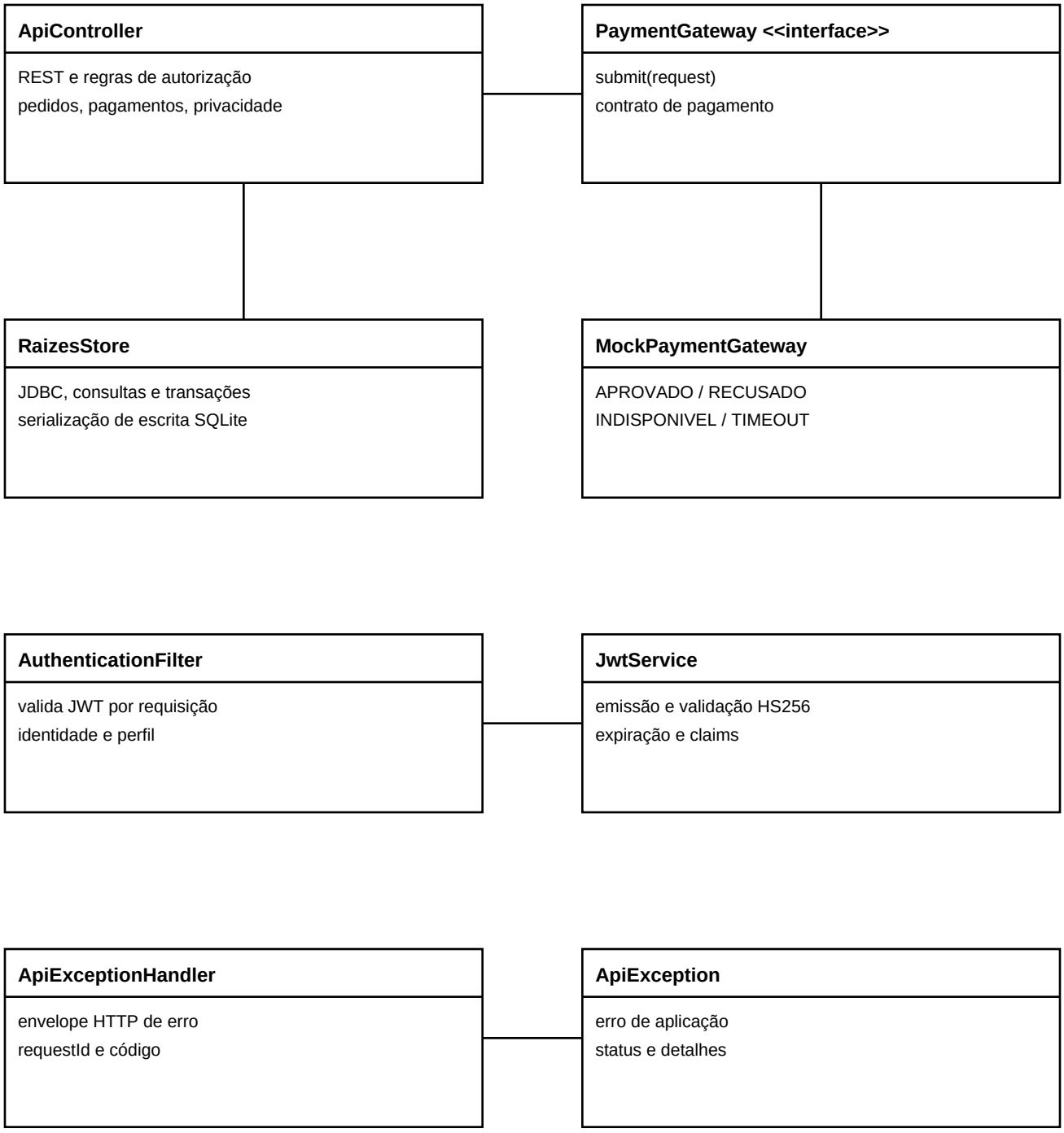


Anexo B - Casos de uso

Visão dos atores e operações; gateway é um adaptador simulado, não uma integração financeira ativa.



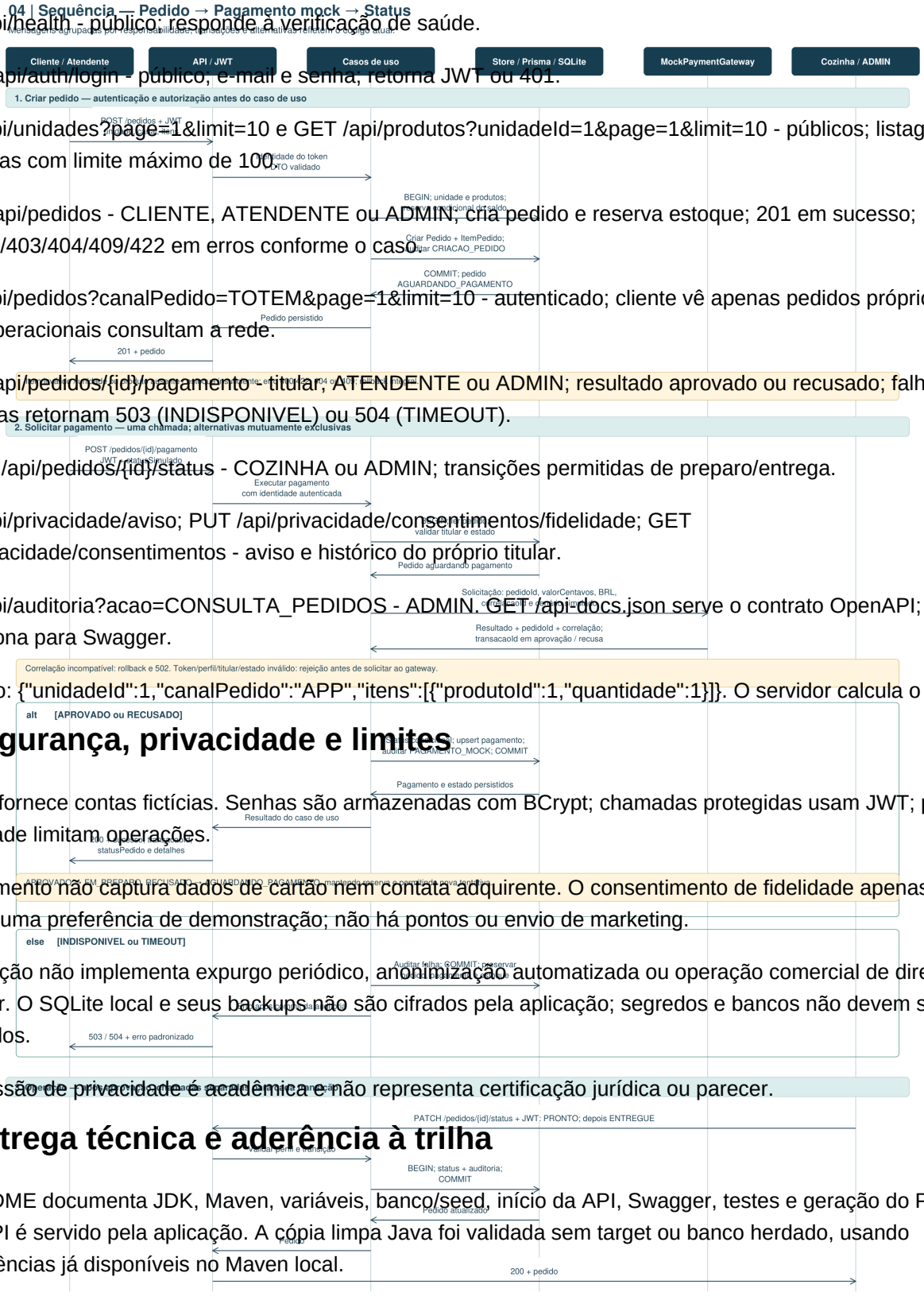
Anexo C - Diagrama de classes Java (visão técnica)



Anexo D - Sequência do fluxo crítico

Visão de pedido, pagamento mock e status. A fonte visual original antecede a versão Java e requer revisão final.

5. API e endpoints



GET /api/health - público; responde à verificação de saúde.

POST /api/auth/login - público, e-mail e senha, retorna JWT ou 401.

GET /api/unidades?page=1&limit=10 e GET /api/produtos?unidadeId=1&page=1&limit=10 - públicos; listagens paginadas com limite máximo de 100.

POST /api/pedidos - CLIENTE, ATENDENTE ou ADMIN; cria pedido e reserva estoque; 201 em sucesso; 400/401/403/404/409/422 em erros conforme o caso.

GET /api/pedidos?canalPedido=TOTEM&page=1&limit=10 - autenticado; cliente vê apenas pedidos próprios; perfis operacionais consultam a rede.

POST /api/pedidos/{id}/pagamento - titular, ATENDENTE ou ADMIN; resultado aprovado ou recusado; falhas simuladas retornam 503 (INDISPONIVEL) ou 504 (TIMEOUT).

PATCH /api/pedidos/{id}/status - COZINHA ou ADMIN; transições permitidas de preparo/entrega.

GET /api/privacidade/aviso; PUT /api/privacidade/consentimentos/fidelidade; GET /api/privacidade/consentimentos - aviso e histórico do próprio titular.

GET /api/auditoria?acao=CONSULTA_PEDIDOS - ADMIN. GET /api-docs.json serve o contrato OpenAPI; / redireciona para Swagger.

Exemplo: {"unidadeId":1,"canalPedido":"APP","itens":[{"produtoId":1,"quantidade":1}]}. O servidor calcula o preço.

6. Segurança, privacidade e limites

O seed fornece contas fictícias. Senhas são armazenadas com BCrypt; chamadas protegidas usam JWT; perfis e titularidade limitam operações.

O pagamento não captura dados de cartão nem contata adquirente. O consentimento de fidelidade apenas registra uma preferência de demonstração; não há pontos ou envio de marketing.

A aplicação não implementa expurgo periódico, anonimização automatizada ou operação comercial de direitos do titular. O SQLite local e seus backups não são cifrados pela aplicação; segredos e bancos não devem ser publicados.

A discussão de privacidade é acadêmica e não representa certificação jurídica ou parecer.

7. Entrega técnica e aderência à trilha

O README documenta JDK, Maven, variáveis, banco/seed, início da API, Swagger, testes e geração do PDF. O OpenAPI é servido pela aplicação. A cópia limpa Java foi validada sem target ou banco herdado, usando dependências já disponíveis no Maven local.

O repositório público e a coleção Postman não foram validados contra a revisão Java corrente; o push ainda não foi feito. Não apresentar a coleção legada como execução comprovada da API Java.

A pasta uninter foi adotada como referência por instrução da discente, embora os próprios PDFs indiquem edição para estudo, omissões e prevalência do original. A adoção não autentica a procedência do material no AVA.

8. Plano de testes e evidências

A validação Maven executada em 11/10/2026 concluiu com BUILD SUCCESS: 18 testes aprovados, sem falhas, erros ou testes ignorados.

Casos positivos cobrem token de autenticação usado nos testes, listagens, pedido criado com itens agrupados, aprovação de pagamento, avanço para PRONTO, consentimento idempotente, seed idempotente e consultas concorrentes.

Casos negativos cobrem credencial/token inválido (401), perfil ou titular indevido (403), dados inválidos (400/422), unidade/produto inexistente (404), estoque insuficiente e conflito de estado (409), gateway (503/504), corpo excessivo (413) e rollback quando falha auditoria.

A evidência é JUnit: 16 métodos de integração em ApiIntegrationTest e 2 testes do gateway em MockPaymentGatewayTest. Os métodos podem conter várias asserções e códigos HTTP. A coleção Postman histórica não foi reexecutada contra Java.

9. Ensaio de desempenho

A medição foi executada contra o JAR Spring Boot empacotado, em loopback, com SQLite isolado e seed local. Vinte workers concorrentes alternaram consultas públicas de produtos e unidades por 60 segundos, usando clientes HTTP persistentes.

Resultado observado: 228.399 requisições em 60,01 segundos; p95 de 11,76 ms; zero falhas e zero respostas HTTP 5xx. Meta local: p95 até 1.000 ms, zero falhas e menos de 1% de respostas 5xx. O cenário atingiu esses limites.

É um ensaio de leituras locais, não de escritas, rede externa, múltiplas instâncias, reinício, restauração ou disponibilidade em produção.

10. Conclusão

A implementação Java entrega o fluxo acadêmico de pedidos, reserva de estoque, pagamento simulado e acompanhamento operacional. Os testes e a medição documentam somente o comportamento observado no ambiente local.

O código executável da API, testes e ferramentas mantidas está em Java 21; a publicação remota e a validação da coleção Postman contra esta revisão continuam pendentes. A cópia limpa Java foi validada. A referência da pasta uninter foi usada conforme orientação da discente, preservada a ressalva indicada pelos próprios arquivos.

Referências

Materiais de referência designados pela discente: PDFs presentes na pasta uninter; os arquivos locais indicam versão editada para estudo.

Projeto Raízes do Nordeste. Documentação técnica, requisitos, privacidade e desempenho. docs/.

Oracle. Java Documentation. docs.oracle.com/en/java/.

Spring. Spring Boot Reference Documentation. docs.spring.io/spring-boot/.

Brasil. Lei nº 13.709, de 14 de agosto de 2018 (LGPD). Texto compilado no Portal da Legislação.